

Bayesian modelling

Markov chain Monte Carlo

Léo Belzile

Last compiled Tuesday Sep 22, 2026

Markov chain Monte Carlo

We consider simulating from a distribution with associated density function proportional to $p(\cdot)$

- using an algorithm that generates a Markov chain ,
- without requiring knowledge of the normalizing factor.

We use a conditional density $q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{\text{cur}})$ to generate proposals from the current value $\boldsymbol{\theta}^{\text{cur}}$.

Metropolis–Hastings algorithm

Starting from an initial value $\boldsymbol{\theta}_0$: for $t = 1, \dots, T$

1. draw a proposal value $\boldsymbol{\theta}_t^* \sim q(\boldsymbol{\theta} \mid \boldsymbol{\theta}_{t-1})$.

2. Compute the acceptance ratio

$$R = \frac{p(\boldsymbol{\theta}_t^*)}{p(\boldsymbol{\theta}_{t-1})} \frac{q(\boldsymbol{\theta}_{t-1} \mid \boldsymbol{\theta}_t^*)}{q(\boldsymbol{\theta}_t^* \mid \boldsymbol{\theta}_{t-1})}$$

3. With probability $\alpha = \min\{R, 1\}$, accept the proposal and set $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_t^*$, otherwise set the value to the previous state, $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1}$.

Theory

The Metropolis–Hastings algorithm satisfies a technical condition (detailed balance) that ensures that the Markov chain generated is reversible.

- Thus, any draw from the posterior will generate a new realization from the posterior.
- Provided the starting value has non-zero probability under the posterior, the chain will converge to the stationarity distribution (albeit perhaps slowly).

Interpretation

- If $R > 1$, the proposal has higher density and we always accept the move.
- If we reject the move, the Markov chain stays at the current value, which induces autocorrelation.
- Since the acceptance probability depends only on the density through ratios, normalizing factors of p and q cancel out.

Symmetric proposals and random walk

If the proposal is symmetric, the ratio of proposal densities is

$$q(\boldsymbol{\theta}_{t-1} \mid \boldsymbol{\theta}_t^*) / q(\boldsymbol{\theta}_t^* \mid \boldsymbol{\theta}_{t-1}) = 1.$$

Common examples include random walk proposals

$$\boldsymbol{\theta}_t^* \leftarrow \boldsymbol{\theta}_{t-1} + \tau Z,$$

where $Z \sim \text{Gauss}(0, 1)$.

Independent proposals

- If we pick instead a global proposal, we must ensure that q samples in far regions (recall rejection sampling), otherwise ...
- Good proposals include heavy tailed distribution such as Student- t with small degrees of freedom,
- centered at the maximum a posteriori $\hat{\boldsymbol{\theta}}$ and
- with scale matrix proportional to $-\mathbf{H}^{-1}(\boldsymbol{\theta}_t^*)$, where $\mathbf{H}(\cdot)$ is the Hessian of the (unnormalized) log posterior.

Upworthy data example

We model the Poisson rates for headlines with questions or not. Our model is

$$Y_i \sim \text{Poisson}(n_i \lambda_i), \quad (i = 1, 2)$$

$$\lambda_1 = \exp(\beta + \kappa)$$

$$\lambda_2 = \exp(\beta)$$

$$\beta \sim \text{Gauss}(\log 0.01, 1.5)$$

$$\kappa \sim \text{Gauss}(0, 1)$$

Implementation details: data and containers

In regression models, scale inputs if possible.

```
1 # Load data
2 data(upworthy_question, package = "hecbayes")
3 # Compute sufficient statistics
4 data <- upworthy_question |>
5   dplyr::group_by(question) |>
6   dplyr::summarize(ntot = sum(impressions),
7                     y = sum(clicks))
8 # Create containers for MCMC
9 niter <- 1e4L
10 chain <- matrix(0, nrow = niter, ncol = 2L)
11 colnames(chain) <- c("beta", "kappa")
12 logpost_v <- numeric(niter)
```

Implementation details: log posterior function

Perform all calculations on the log scale to avoid numerical overflow!

```
1 # Code log posterior as sum of log likelihood and log prior
2 loglik <- function(par, counts = data$y, offset = data$ntot, ...){
3   lambda <- exp(c(par[1] + log(offset[1]),
4                   par[1] + par[2] + log(offset[2])))
5   sum(dpois(x = counts, lambda = lambda, log = TRUE))
6 }
7 # Note common signature of function
8 logprior <- function(par, ...){
9   dnorm(x = par[1], mean = log(0.01), sd = 1.5, log = TRUE) +
10   dnorm(x = par[2], log = TRUE)
11 }
12 logpost <- function(par, ...){
13   loglik(par, ...) + logprior(par, ...)
14 }
```

Implementation details: proposals

Use good starting values for your Markov chains, such as maximum a posteriori.

```
1 # Compute maximum a posteriori (MAP)
2 map <- optim(
3   par = c(-4, 0.07),
4   fn = logpost,
5   control = list(fnscale = -1),
6   offset = data$ntot,
7   counts = data$,
8   hessian = TRUE)
9 # Use MAP as starting value
10 cur <- map$par
11 # Keep track of log posterior to reduce calculations
12 logpost_cur <- logpost(cur)
13 # Proposal covariance
14 cov_map <- -2*solve(map$hessian)
15 chol <- chol(cov_map)
```

Implementation details: Metropolis–Hastings

Use seed for reproducibility, do not compute posterior twice, compute log of acceptance ratio.

```
1  set.seed(80601)
2  naccept <- 0L
3  for(i in seq_len(niter)){
4    # Multivariate normal proposal - symmetric random walk
5    prop <- c(rnorm(n = 2) %*% chol + cur)
6    logpost_prop <- logpost(prop)
7    logR <- logpost_prop - logpost_cur
8    if(logR > -rexp(1)){# accept move
9      cur <- prop
10     logpost_cur <- logpost_v[i] <- logpost_prop
11     naccept <- naccept + 1L
12   }
13   logpost_v[i] <- logpost_cur
14   chain[i,] <- cur
15 }
```

Implementation details: analysis of output

```
1 # Posterior summaries
2 summary(coda::as.mcmc(chain))
```

1. Empirical mean and standard deviation for each variable, plus standard error of the mean:

	Mean	SD	Naive SE	Time-series SE
beta	-4.51268	0.001697	1.697e-05	6.176e-05
kappa	0.07075	0.002033	2.033e-05	9.741e-05

2. Quantiles for each variable:

	2.5%	25%	50%	75%	97.5%
beta	-4.51591	-4.51385	-4.51273	-4.51154	-4.50929
kappa	0.06673	0.06933	0.07077	0.07212	0.07463

Metropolis–Hastings algorithm

Starting from an initial value $\boldsymbol{\theta}_0$:

1. draw a proposal value $\boldsymbol{\theta}_t^* \sim q(\boldsymbol{\theta} \mid \boldsymbol{\theta}_{t-1})$.
2. Compute the acceptance ratio

$$R = \frac{p(\boldsymbol{\theta}_t^*)}{p(\boldsymbol{\theta}_{t-1})} \frac{q(\boldsymbol{\theta}_{t-1} \mid \boldsymbol{\theta}_t^*)}{q(\boldsymbol{\theta}_t^* \mid \boldsymbol{\theta}_{t-1})}$$

3. With probability $\min\{R, 1\}$, accept the proposal and set $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_t^*$, otherwise set the value to the previous state, $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1}$.

Calculations

We compute the log of the acceptance ratio, $\ln R$, to avoid numerical overflow, with the log posterior difference

$$\ln \left\{ \frac{p(\boldsymbol{\theta}_t^*)}{p(\boldsymbol{\theta}_{t-1})} \right\} = \ell(\boldsymbol{\theta}_t^*) + \ln p(\boldsymbol{\theta}_t^*) - \ell(\boldsymbol{\theta}_{t-1}) - \ln p(\boldsymbol{\theta}_{t-1})$$

Compare the value of $\ln R$ (if less than zero) to $\log(U)$, where $U \sim \text{unif}(0, 1)$.

What proposal?

The *independence* Metropolis–Hastings uses a global proposal q which does not depend on the current state (typically centered at the MAP)

- This may be problematic with multimodal targets.

The Gaussian random walk takes $\boldsymbol{\theta}_t^* = \boldsymbol{\theta}_{t-1} + \sigma_p Z$, where $Z \sim \text{Gauss}(0, 1)$ and σ_p is the proposal standard deviation. *Random walks* allow us to explore the space.

Assessing convergence

How do we assess convergence of a MCMC algorithm?

- the algorithm implementation must be correct,
- the chain must have converged to the target posterior.
- the effective sample size must be sufficiently large for inference.

Tracking the log posterior

One can store (separately) at each iteration the values of the

- log likelihood
- log prior

These can be used for prior or likelihood sensitivity
([Kallioinen et al., 2023](#))

Blank run

To check that the algorithm is correctly implemented:

- remove the log likelihood component
- run the algorithm with just the prior

The resulting draws should be drawn from the prior if the latter is proper ([Green, 2001, p. 55](#)).

Burn in

We are guaranteed to reach stationarity with Metropolis–Hastings eventually, but it may take a large number of iterations...

One should discard initial draws during a **burn in** or warmup period if the chain has not reached stationarity. Ideally, use good starting value to reduce waste.

We can also use the warmup period to adapt the variance of the proposal.

Goldilock principle and proposal variance

Mixing of the chain requires just the right variance (not too small nor too large).

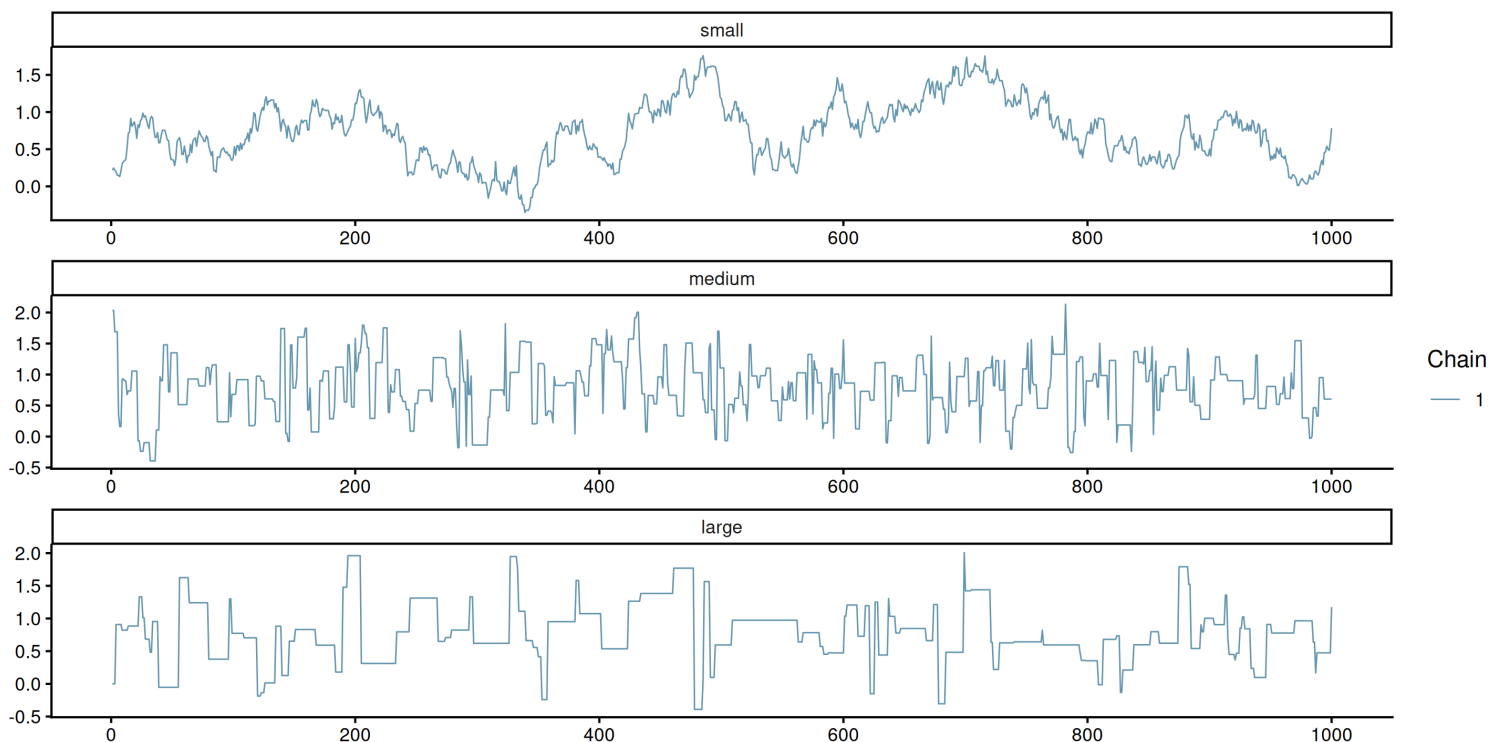


Figure 1: Example of traceplot with proposal variance that is too small (top), adequate (middle) and too large (bottom).

Correlograms for Goldilock

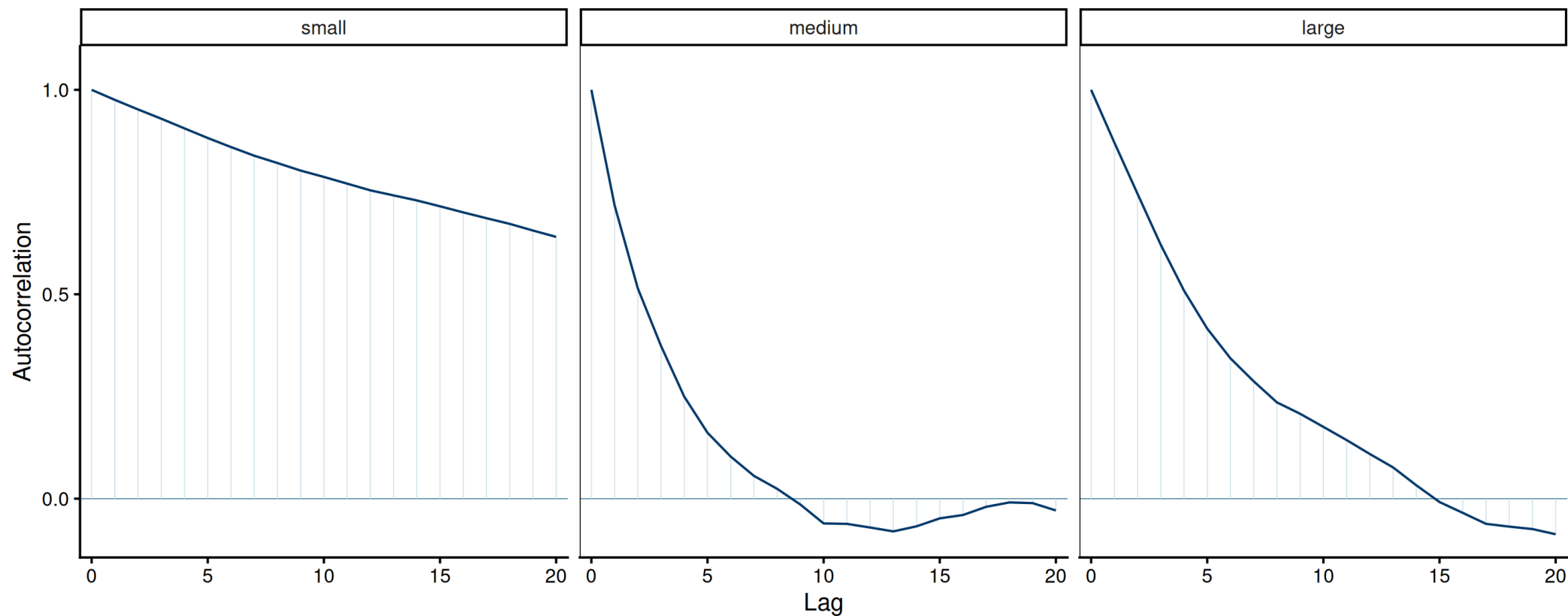


Figure 2: Correlogram for the three Markov chains.

Tuning Markov chain Monte Carlo

- Outside of starting values, the variance of the proposal has a huge impact on the asymptotic variance.
- We can adapt the variance during warmup by increasing/decreasing proposal variance (if acceptance rate is too large/small).
- We can check this via the acceptance rate (how many proposals are accepted).

Optimal acceptance rates

The following rules were derived for Gaussian targets under idealized situations.

- In 1D, rule of thumb is an acceptance rate of 0.44 is optimal, and this ratio decreases to 0.234 when $D \geq 2$ (Sherlock, 2013) for random walk Metropolis–Hastings.
- Proposals for D -variate update should have proposal variance of roughly $(2.38^2 / d) \times \Sigma$, where Σ is the posterior variance.
- For MALA (see later), we get 0.574 rather than 0.234

Block update or one parameter at a time?

As with any accept-reject, proposals become inefficient when the dimension D increase.

This is the **curse of dimensionality**.

Updating parameters in turn

- increases acceptance rate (with clever proposals),
- but also leads to more autocorrelation between parameters

Solutions for strongly correlated coefficients

- Reparametrize the model to decorrelate variables (orthogonal parametrization).
- Block updates: draw correlated parameters together
 - using the chain history to learn the correlation, if necessary

Parameter transformation

Parameters may be bounded, e.g. $\theta_i \in [a, b]$.

- We can ignore this and simply discard proposals outside of the range, by setting the log posterior at $-\infty$ outside $[a, b]$
- We can do a transformation, e.g., $\log \theta_i$ if $\theta_i > 0$ and perform a random walk on the unconstrained space: don't forget Jacobians for $q(\cdot)$!
- Another alternative is to use truncated proposals (useful with more complex algorithms like MALA)

Efficient proposals: MALA

The Metropolis-adjusted Langevin algorithm (MALA) uses a Gaussian random walk proposal

$$\boldsymbol{\theta}_t^* \sim \text{Gauss}\{\mu(\boldsymbol{\theta}_{t-1}), \tau^2 \mathbf{A}\},$$

with mean

$$\mu(\boldsymbol{\theta}_{t-1}) = \boldsymbol{\theta}_{t-1} + \mathbf{A}\eta\nabla \log p(\boldsymbol{\theta}_{t-1} \mid \mathbf{y}),$$

and variance $\tau^2 \mathbf{A}$, for some mass matrix $\mathbf{A}$, tuning parameter $\tau > 0$.

The parameter $\eta < 1$ is a learning rate. This is akin to a Newton algorithm, so beware if you are far from the mode (where the gradient is typically large)!

Higher order proposals

For a single parameter update θ , a Taylor series expansion of the log posterior around the current value suggests using as proposal density a Gaussian approximation with (Rue & Held, 2005)

- mean $\mu_{t-1} = \theta_{t-1} - f'(\theta_{t-1})/f''(\theta_{t-1})$ and
- precision $\tau^{-2} = -f''(\theta_{t-1})$

We need $f''(\theta_{t-1})$ to be negative!

This gives **local adaption** relative to MALA (global variance).

Higher order and moves

For MALA and cie., we need to compute the density of the proposal also for the reverse move for the expansion starting from the proposal $\mu(\boldsymbol{\theta}_t^*)$.

These methods are more efficient than random walk Metropolis–Hastings, but they require the gradient and the hessian (can be obtained analytically using autodiff, or numerically).

Modelling individual headlines of Upworthy example

The number of conversions `nclick` is binomial with sample size $n_i = \text{nimpression}$.

Since n_i is large, the sample average `nclick/nimpression` is approximately Gaussian, so write

$$Y_i \sim \text{Gauss}(\mu, \sigma^2/n_i)$$

$$\mu \sim \text{trunc. Gauss}(0.01, 0.1^2, 0, 1)$$

$$\sigma \sim \text{expo}(0.7)$$

MALA: data set-up

```
1 data(upworthy_question, package = "hecbayes")
2 # Select data for a single question
3 qdata <- upworthy_question |>
4   dplyr::filter(question == "yes") |>
5   dplyr::mutate(y = clicks/impressions,
6                 no = impressions)
```


MALA: define functions

```
1 # Create functions with the same signature (...) for the algorithm
2 logpost <- function(par, data, ...){
3   mu <- par[1]; sigma <- par[2]
4   no <- data$no
5   y <- data$y
6   if(isTRUE(any(sigma <= 0, mu < 0, mu > 1))){
7     return(-Inf)
8   }
9   dnorm(x = mu, mean = 0.01, sd = 0.1, log = TRUE) +
10  dexp(sigma, rate = 0.7, log = TRUE) +
11  sum(dnorm(x = y, mean = mu, sd = sigma/sqrt(no), log = TRUE))
12 }
```

MALA: compute gradient of log posterior

```
1 logpost_grad <- function(par, data, ...){  
2   no <- data$no  
3   y <- data$y  
4   mu <- par[1]; sigma <- par[2]  
5   c(sum(no*(y-mu))/sigma^2 -(mu - 0.01)/0.01,  
6     -length(y)/sigma + sum(no*(y-mu)^2)/sigma^3 -0.7  
7   )  
8 }
```

MALA: compute maximum a posteriori

```
1 # Starting values - MAP
2 map <- optim(
3   par = c(mean(qdata$y), 0.5),
4   fn = function(x){-logpost(x, data = qdata)},
5   gr = function(x){-logpost_grad(x, data = qdata)},
6   hessian = TRUE,
7   method = "BFGS")
8 # Check convergence
9 logpost_grad(map$par, data = qdata)
```

MALA: starting values and mass matrix

```
1 # Set initial parameter values
2 curr <- map$par
3 # Compute a mass matrix
4 Amat <- solve(map$hessian)
5 # Cholesky root - for random number generation
6 cholA <- chol(Amat)
```

MALA: containers and setup

```
1 # Create containers for MCMC
2 B <- 1e4L # number of iterations
3 warmup <- 1e3L # adaptation period
4 npar <- 2L
5 prop_sd <- rep(1, npar) # tuning parameter
6 chains <- matrix(nrow = B, ncol = npar)
7 damping <- 0.8
8 acceptance <- attempts <- 0
9 colnames(chains) <- names(curr) <- c("mu", "sigma")
10 # Proposal variance proportional to inverse hessian at MAP
11 prop_var <- diag(prop_sd) %*% Amat %*% diag(prop_sd)
```

MALA: sample proposal with Newton step

```
1 for(i in seq_len(B + warmup)){
2   ind <- pmax(1, i - warmup)
3   # Compute the proposal mean for the Newton step
4   prop_mean <- c(curr + damping *
5     Amat %*% logpost_grad(curr, data = qdata))
6   # prop <- prop_sd * c(rnorm(npar) %*% cholA) + prop_mean
7   prop <- c(mvtnorm::rmvnorm(
8     n = 1,
9     mean = prop_mean,
10    sigma = prop_var))
11 # [...]
```

MALA: reverse step

```
1  # Compute the reverse step
2  curr_mean <- c(prop + damping *
3    Amat %*% logpost_grad(prop, data = qdata))
4  # log of ratio of bivariate Gaussian densities
5  logmh <- mvtnorm::dmvnorm(
6    x = curr, mean = prop_mean,
7    sigma = prop_var,
8    log = TRUE) -
9    mvtnorm::dmvnorm(
10     x = prop,
11     mean = curr_mean,
12     sigma = prop_var,
13     log = TRUE) +
14  logpost(prop, data = qdata) -
15  logpost(curr, data = qdata)
```

MALA: Metropolis–Hastings ratio

```
1  if(logmh > log(runif(1))){  
2    curr <- prop  
3    acceptance <- acceptance + 1L  
4  }  
5  attempts <- attempts + 1L  
6  # Save current value  
7  chains[ind,] <- curr
```


MALA: adaptation

```
1  if(i %% 100 & i < warmup){
2    # Check acceptance rate and increase/decrease variance
3    out <- hecbayes::adaptive(
4      attempts = attempts, # counter for number of attempts
5      acceptance = acceptance,
6      sd.p = prop_sd, #current proposal standard deviation
7      target = 0.574) # target acceptance rate
8    prop_sd <- out$sd # overwrite current std.dev
9    acceptance <- out$acc # if we change std. dev, this is set to zero
10   attempts <- out$att # idem, otherwise unchanged
11   prop_var <- diag(prop_sd) %*% Amat %*% diag(prop_sd)
12 }
13 } # End of MCMC for loop
```

Strategies

Many diagnostics require running multiple chains

- check within vs between variance,
- determine whether they converge to the same target.

Correct implementation

We can generate artificial data to check the procedure.

Simulation-based calibration ([Talts et al., 2020](#)) proceeds with, in order

1. $\boldsymbol{\theta}_0 \sim p(\boldsymbol{\theta})$,
2. $\mathbf{y}_0 \sim p(\mathbf{y} \mid \boldsymbol{\theta}_0)$,
3. $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_B \sim p(\boldsymbol{\theta} \mid \mathbf{y}_0)$.

Simulation-based calibration

- Conditional on the simulated $\mathbf{y}$, the distribution of $\boldsymbol{\theta}_0$ is the same as that of $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_B$.
- We do a dimension reduction step taking the test function $t(\cdot)$ to get the rank of the prior draw among the posterior ones, breaking ties at random if any.
- These steps are repeated K times, yielding K test functions $T_1, \dots, T_K$. We then test for uniformity using results from Säilynoja et al. (2022).

Breaking down the Markov chain

We distinguish between three phases

- **burn in** period: initial draws allowing the algorithm to converge to its stationary distribution (discarded)
- **warmup** adaptation period: tuning period for the proposal std. deviation, etc. (discarded)
- **sampling** period: draws post burn in and warmup that are kept for inference

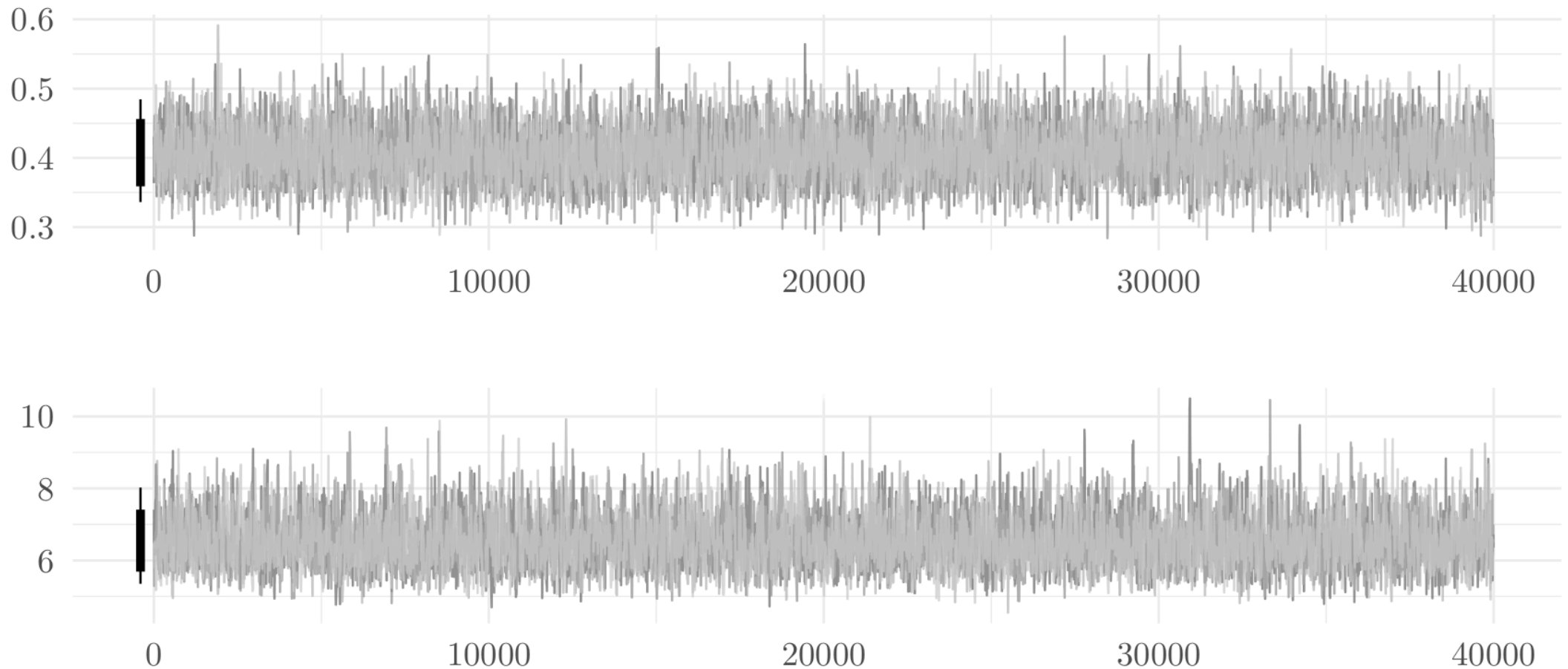
We can optionally **thin** by keeping one every k iterations from the sampling period to reduce the storage.

Visual diagnostic: trace plots

Display the Markov chain sample path as a function of the number of iterations.

- Ideally, run multiple chains to see if they converge to the same mode (for multimodal behaviour).
- Markov chains should look like a fat hairy caterpillar!
- Check the [bayesplot](#) and [coda](#) R packages (trace plot, trace rank, correlograms, marginal densities, etc.)

Checking convergence with multiple chains



Four healthy parallel chains for parameters.

Trace rank plot

A **trace rank** plot compares the rank of the values of the different chain at a given iteration.

- With good mixing, the ranks should switch frequently and be distributed uniformly across integers.

Effective sample size

Are my chains long enough to compute reliable summaries?

Compute the sample size we would have with independent draws by taking

$$\text{ESS} = \frac{B}{\left\{1 + 2 \sum_{t=1}^{\infty} \rho_t\right\}}$$

where ρ_t is the lag t autocorrelation.

The relative effective sample size is simply ESS/B : small values indicate pathological or inefficient samplers.

How many samples?

We want our average estimate to be reliable!

- We probably need **ESS** to be several hundred
- We can estimate the variance of the target to know the precision
- (related question: how many significant digits to report?)

Lack of stationarity

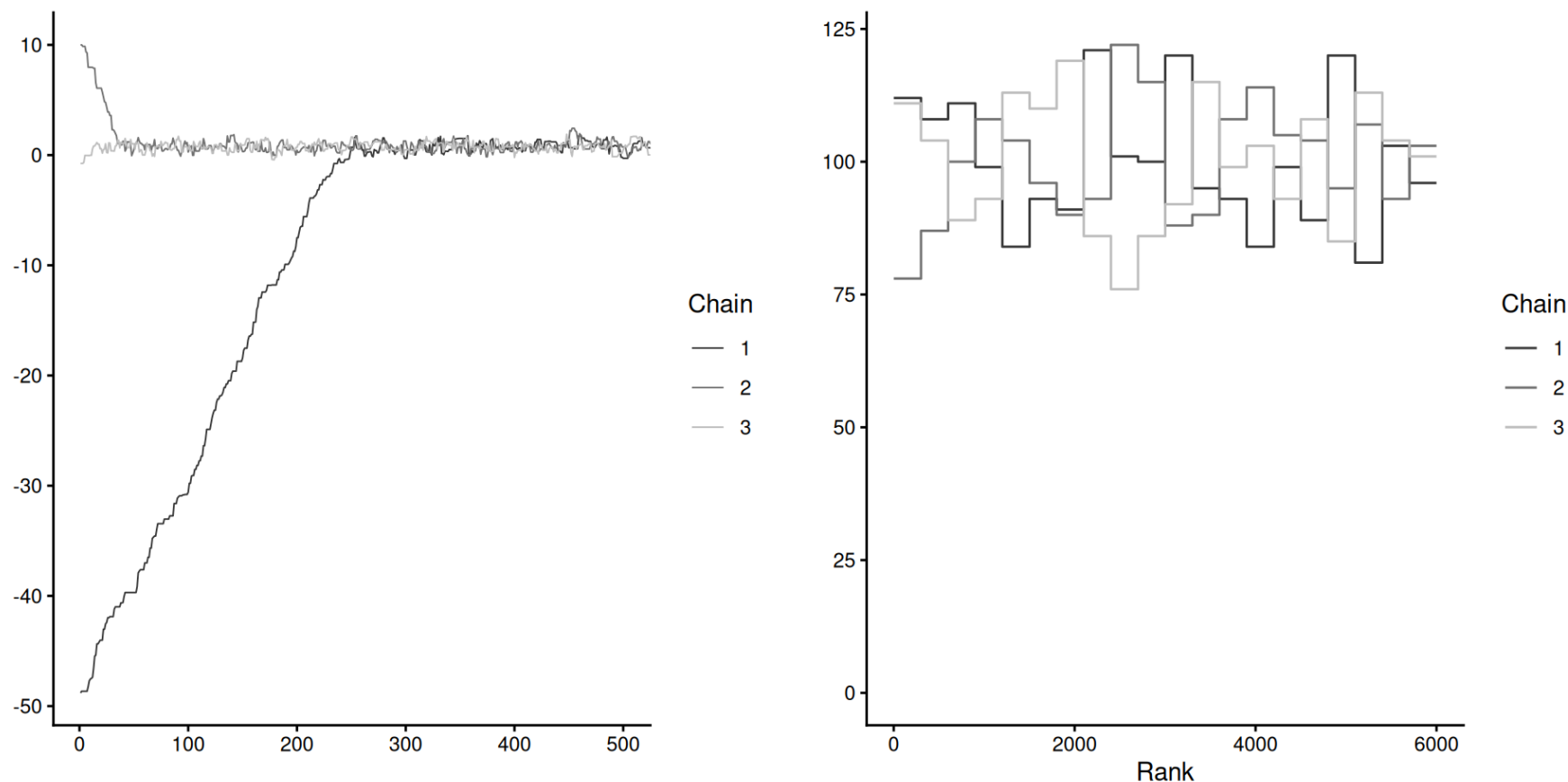


Figure 3: Traceplots of three Markov chains for the same target with different initial values for the first 500 iterations (left) and trace rank plot after discarding these (right). The latter is indicative of the speed of mixing.

Gelman–Rubin diagnostic

Suppose we run M chains for B iterations, post burn in.

Denote by θ_{bm} the b th draw of the m th chain, we compute the global average

$$\bar{\theta} = \frac{1}{BM} \sum_{b=1}^B \sum_{m=1}^M \theta_{bm}$$

and similarly the chain-specific sample average and variances, respectively $\bar{\theta}_m$ and $\hat{\sigma}_m^2$ ($m = 1, \dots, M$).

Sum of square decomposition

The between-chain variance and within-chain variance estimator are

$$Va_{\text{between}} = \frac{B}{M-1} \sum_{m=1}^M (\bar{\theta}_m - \bar{\theta})^2$$

$$Va_{\text{within}} = \frac{1}{M} \sum_{m=1}^m \hat{\sigma}_m^2$$

Potential scale reduction statistic

The Gelman–Rubin diagnostic, denoted $\hat{R}$, is obtained by running multiple chains and considering the difference between within-chain and between-chains variances,

$$\hat{R} = \left(\frac{\mathbf{Va}_{\text{within}}(B - 1) + \mathbf{Va}_{\text{between}}}{B\mathbf{Va}_{\text{within}}} \right)^{1/2}$$

Any value of $\hat{R}$ larger 1 is indicative of problems of convergence.

Bad chains

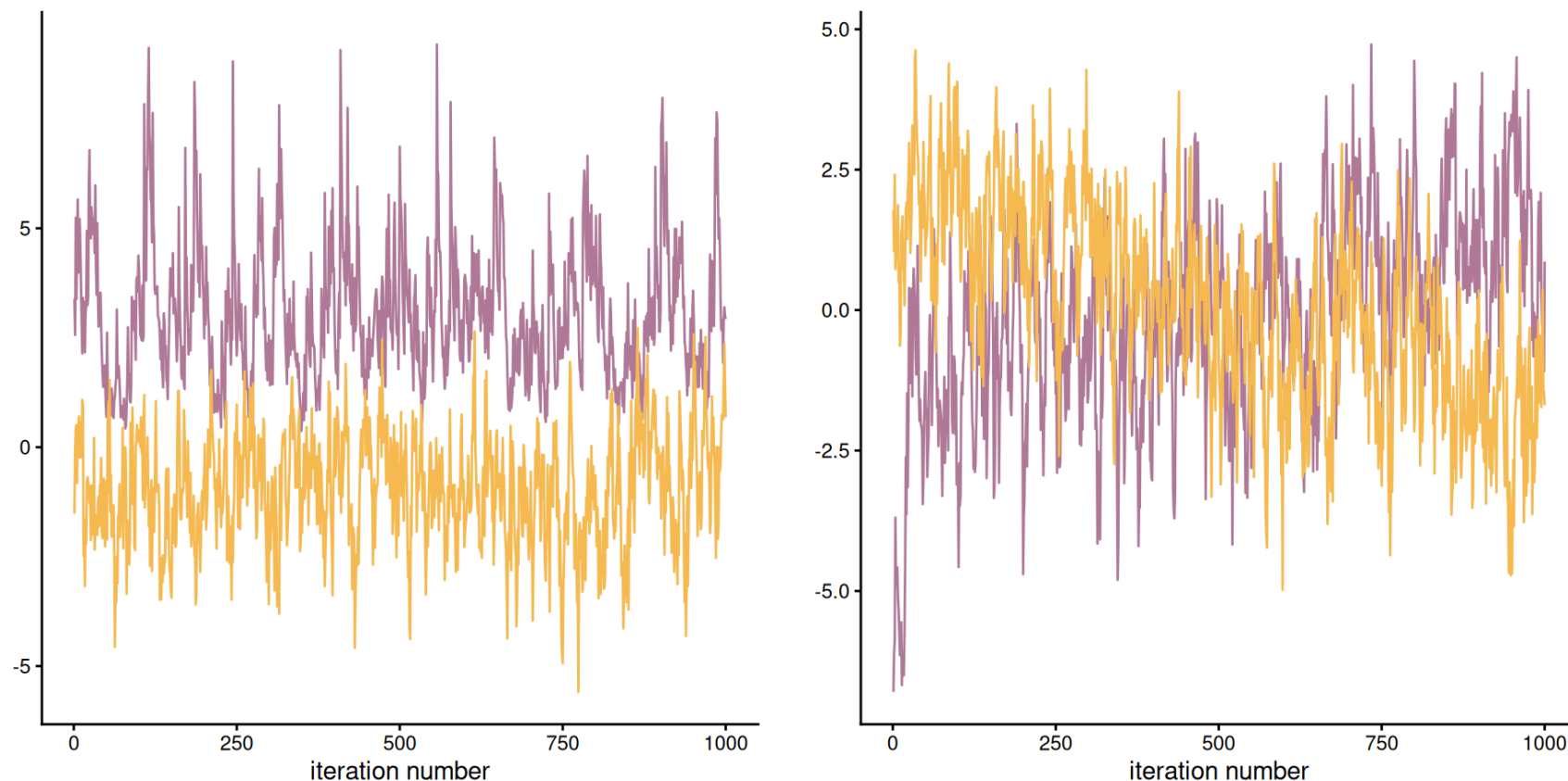


Figure 4: Two pairs of Markov chains: the top ones seem stationary, but with different modes and $\hat{R} \approx 3.4$. The chains on the right hover around zero, but do not appear stable, with $\hat{R} \approx 1.6$.

One chain or multiple chains?

Generally, it is preferable to run a single chain for a longer period than run multiple chains sequentially

- there is a cost to initializing multiple times with different starting values since we must discard initial draws.
- but with parallel computations, multiple chains are more frequent nowadays.
- multiple diagnostics require running several chains.

Posterior predictive checks

1. For each of the B draws from the posterior, simulate n observations from the posterior predictive $p(\tilde{\mathbf{y}} \mid \mathbf{y})$
2. For each replicate, compute a summary statistics (median, quantiles, std. dev., etc.)
3. Compare it with the same summary computed for the sample $\mathbf{y}$.

Posterior predictive checks

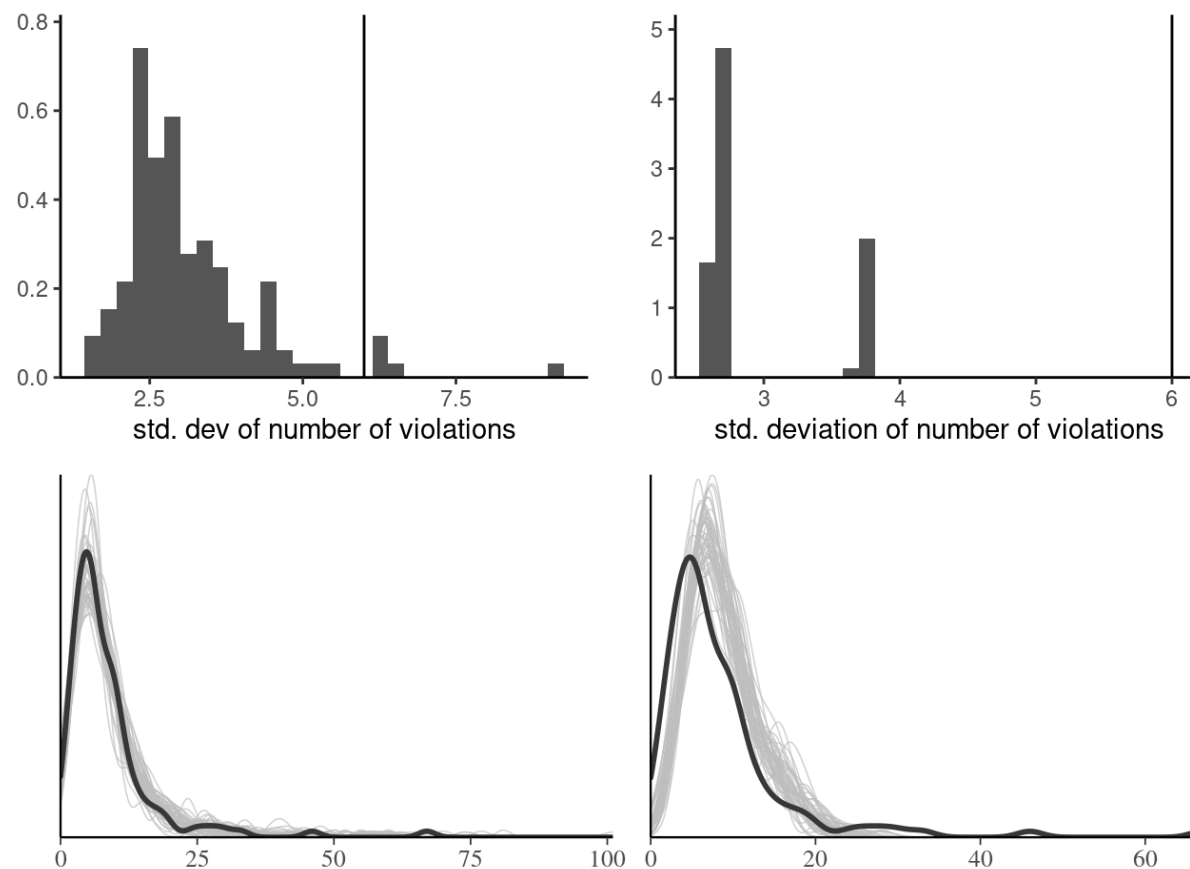


Figure 5: Posterior predictive checks for the standard deviation (top) and density of posterior draws (bottom) for hierarchical Poisson model with individual effects (left) and simpler model with only conditions (right).

Log pointwise predictive density

Consider the expected value of the observation-wise log density with respect to the posterior distribution $p(\boldsymbol{\theta} \mid \mathbf{y})$,

$$\text{LPPD}_i = \mathbf{E}_{\boldsymbol{\theta} \mid \mathbf{y}} \{ \log p(y_i \mid \boldsymbol{\theta}) \},$$

The higher the value of LPPD_i , the better the fit for that observation.

Widely available information criterion

To build an information criterion, we add a penalization factor that approximates the effective number of parameters in the model, with

$$n\text{WAIC} = - \sum_{i=1}^n \text{LPPD}_i + \sum_{i=1}^n \text{Va}_{\boldsymbol{\theta}|\mathbf{y}} \{ \log p(y_i | \boldsymbol{\theta}) \}$$

where we use again the empirical variance to compute the rightmost term.

Smaller values of WAIC are better.

Pseudo-code for WAIC

Evaluate the log likelihood for each posterior draw and each observation.

```
1 #' WAIC
2 #' @param loglik_pt B by n matrix of pointwise log likelihood
3 WAIC <- function(loglik_pt){
4   -mean(apply(loglik_pt, 2, mean)) + mean(apply(loglik_pt, 2, var))
5 }
```

Bayesian leave-one-out cross validation

In Bayesian setting, we can use the leave-one-out predictive density

$$p(y_i \mid \mathbf{y}_{-i})$$

as a measure of predictive accuracy.

We can use importance sampling to approximate the latter.

Requirement: need to keep track of the log likelihood of each observation for each posterior draw ($B \times n$ values).

LOO-CV diagnostics

We can draw B samples from $p(\tilde{y} \mid \mathbf{y}_{-i})$ and compute the rank of y_i .

Under perfect calibration, ranks should be uniform.

Leave-one-out with quantile-quantile plots

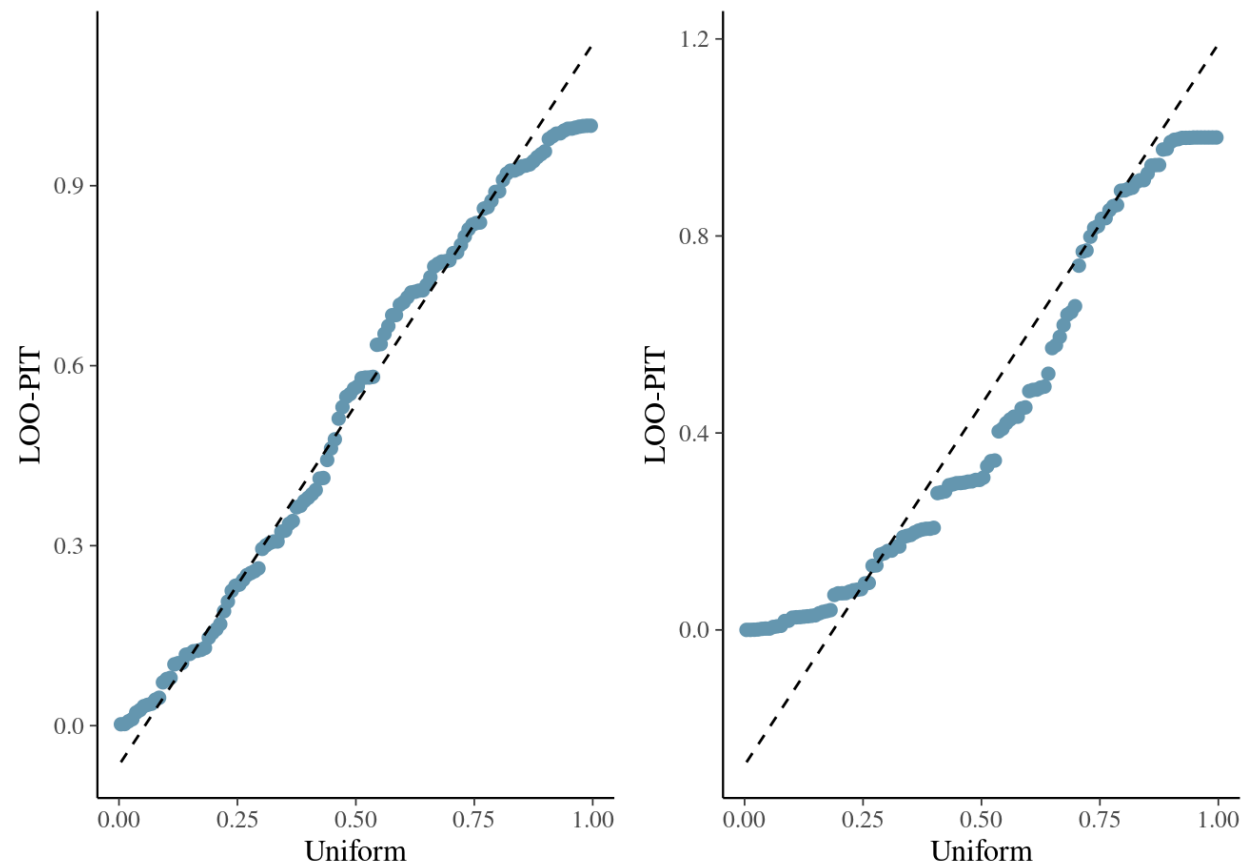


Figure 6: Quantile-quantile plots based on leave-one-out cross validation for model for the hierarchical Poisson model fitted to the Upworthy data with the individual random effects (left) and without (right).

Deviance information criterion

The **deviance** information criterion of Spiegelhalter et al. (2002) is

$$\text{DIC} = -2\ell(\tilde{\boldsymbol{\theta}}) + 2p_D$$

where p_D is the posterior expectation of the deviance relative to the point estimator of the parameter $\tilde{\boldsymbol{\theta}}$ (e.g., the maximum a posteriori or the posterior mean)

$$p_D = \mathbf{E}\{D(\boldsymbol{\theta}, \tilde{\boldsymbol{\theta}}) \mid \mathbf{y}\} = \int 2\{\ell(\tilde{\boldsymbol{\theta}}) - \ell(\boldsymbol{\theta})\}p(\boldsymbol{\theta} \mid \mathbf{y})d\boldsymbol{\theta}$$

Criticism of DIC

- The DIC can be easily evaluated by keeping track of the log likelihood evaluated at each posterior draw from a Markov chain Monte Carlo algorithm.
- The penalty term p_D is however not invariant to reparametrizations.
- A Gaussian approximation to the MLE under suitable regularity conditions shows that the DIC is equivalent in large samples to AIC.

Summary (1/4)

Summary (2/4)

- Good initial starting values can be obtained by computing maximum a posteriori estimates.
- Initializing multiple chains at different starting values can be used to check convergence to the stationary distribution.
- Mixing will improve if strongly correlated parameters are sampled together.

Summary (3/4)

Summary (4/4)

- Metropolis-adjusted Langevin algorithm (MALA) uses the gradient information to inform the proposal; it is akin to a Newton step.
- The detailed balance requires a function g such that $g(r) = rg(1/r)$. Taking $g(r) = \min(1, r)$ as in Metropolis–Hasting rule leads to the lowest asymptotic variance (Peskun, 1973).

References

- Green, P. J. (2001). A primer on Markov chain Monte Carlo. *Monographs on Statistics and Applied Probability*, 87, 1–62.
- Kallioinen, N., Paananen, T., Bürkner, P.-C., & Vehtari, A. (2023). Detecting and diagnosing prior and likelihood sensitivity with power-scaling. *Statistics and Computing*, 34(1), 57. <https://doi.org/10.1007/s11222-023-10366-5>
- Neal, R. M. (2011). MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. Jones, & X. L. Meng (Eds.), *Handbook of Markov chain Monte Carlo* (pp. 113–162). CRC Press. <https://doi.org/10.1201/b10905-5>
- Peskun, P. H. (1973). Optimum Monte-Carlo sampling using Markov chains. *Biometrika*, 60(3), 607–612. <https://doi.org/10.1093/biomet/60.3.607>
- Rue, H., & Held, L. (2005). *Gaussian Markov random fields: Theory and applications* (p. 280). CRC Press.
- Säilynoja, T., Bürkner, P.-C., & Vehtari, A. (2022). Graphical test for discrete uniformity and its applications in goodness-of-fit evaluation and multiple sample comparison. *Statistics and Computing*, 32(2), 32. <https://doi.org/10.1007/s11222-022-10090-6>

- Sherlock, C. (2013). Optimal scaling of the random walk Metropolis: General criteria for the 0.234 acceptance rule. *Journal of Applied Probability*, 50(1), 1–15.
<https://doi.org/10.1239/jap/1363784420>
- Spiegelhalter, D. J., Best, N. G., Carlin, B. P., & Van Der Linde, A. (2002). Bayesian measures of model complexity and fit. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 64(4), 583–639. <https://doi.org/10.1111/1467-9868.00353>
- Talts, S., Betancourt, M., Simpson, D., Vehtari, A., & Gelman, A. (2020). *Validating Bayesian inference algorithms with simulation-based calibration*.
<https://doi.org/10.48550/arXiv.1804.06788>